Document 14 — Debugging Lab (covers everything)

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Prerequisites

Docs 01-13 (this practices everything you've built, together).

How to Read & Practice This Document

- **What:** a repeatable way to figure out failures, instead of guessing.
- **Why:** real AI systems fail in layered, unclear ways — changing things randomly until it works doesn't scale past small projects.
- **When:** every single time something breaks, from here through your real job.
- **How to practice:** this document is live and interactive on purpose — there's no solo build task. Read the method below, have Projects 1-4 working, then say **START DOCUMENT 14** and work through each planted bug using the steps below, out loud, before I confirm or correct your answer.

Jump to: Core Concepts · Practice Exercises

The Story — what this document is actually building

Every document before this one added a new skill: functions and config (Doc01), API calls, tools, RAG, LangGraph, multi-agent orchestration, production hardening, and a real test suite (Doc13). Each one, on its own, worked by the time you moved on. This document is where all of that gets stress-tested at once — because a real system doesn't fail one clean layer at a time, it fails in ways that *look* like they could be any of five different things at once.

First, you need a fixed way to tell a symptom fix from a real fix. Silencing an error with a bare `try/except`, or hardcoding a value that happened to fail once, makes the failure in front of you disappear — without you ever learning why it happened, which means it comes back later, usually more confusing the second time.

Second, you need to be able to read an error trace without panicking — it's not noise, it's an ordered map of exactly which calls were running when things broke, and reading it from the bottom up gets you to the real spot faster than scanning it for anything that looks familiar.

Third, you need a *reliable* way to reproduce a failure before you can test any guess about it — a fix you can't verify isn't really a fix, it's a hope.

Fourth, you need to check failures layer by layer, starting low (was the right tool even called? was its input correct?) instead of starting at the final answer — because in a multi-agent system, the final answer is the layer furthest from the actual cause, and the least useful place to start looking.

That's the whole story: this document doesn't teach new libraries or new code — it teaches **the Method**, a fixed sequence (Observe → Reproduce → Guess → Test → Real Cause → Fix → Test Again → Prevent) applied to real bugs planted into the real projects you already built. There's no Build Task here on purpose — this document *is* the practice, run live, against Projects 1-4, until debugging this way is a habit instead of a checklist.

The Method (learned here, used everywhere after)

```
Observe → Reproduce → Form a Guess → Test the Guess
   → Find the Real Cause → Fix → Test Again → Prevent It Happening Again
```

No "just Google the error." Every planted bug below gets worked through this way, out loud, before I confirm or correct your answer.

Core Concepts (read this first — everything you need is here)

Why "real cause" and "just the symptom" are different things

A **symptom** is what you can see: an error, a crash, a wrong answer. The **real cause** (also called the "root cause") is the reason that symptom happened. A symptom fix makes the failure you see stop right now — for example, a `try/except` that swallows the error, or a hardcoded value that hides one bad case. It does not deal with *why* the failure happened, so the same problem comes back later, often in a more confusing form. A real-cause fix changes the actual reason, and you check it by understanding the failure, not only by seeing the error go away.

Why this difference is the whole point of the method: it is very easy to change code at random until one test passes, call it "fixed", and leave the real bug in place — only better hidden. In AI systems this is even more dangerous. A hidden bug often does not crash. It turns into a *believable wrong answer*: the agent says "it is 0°C in Lahore" with full confidence. A crash gets noticed in minutes. A believable wrong answer can reach users for weeks. The method exists to stop this.

When is a quick symptom fix OK, and when is it not?

Situation	What to do	Why	Small example
Production is down and users are blocked right now	A short-term symptom fix is OK, **but** log it and open a task for the real cause	Users come first, but the bug is still there	Turn off the broken tool, and log `logger.error("weather tool disabled, see ticket 42")`
You are fixing a bug during normal work	Find and fix the real cause	A hidden bug returns later, and is harder to find the second time	Fix the bad API key, not the `KeyError` it caused
The error comes from bad outside data you truly cannot control	Handle it on purpose, **and** make it visible	This is a real fix only if you know the cause and you log it	`except ValueError: logger.warning("bad row skipped: %s", row)` (same rule as Doc01)
You "fixed" it but cannot explain why the bug happened	You are not done — keep looking	If you cannot explain it, you only changed the symptom	"I added `.get()` and now it works" is not an explanation

Where you'll meet it: in every planted bug in this document. After this, you meet it every time a real system breaks: a support chatbot (Project 1) that sends the wrong category, a RAG app over company documents (Project 3) that gives a wrong answer, a scheduled job that "succeeds" with empty data. It matters most in multi-agent systems (Doc11, Project 4): one agent's hidden symptom fix (return an empty string instead of crashing) becomes the next agent's "bad input", and the real cause is now two agents away from where you see the problem.

How it works — ask "why?" until you reach something you can change: start from the symptom and keep asking "why did that happen?". Stop when the answer is something in your code, data, or config that you can actually fix.

```
Symptom:  the agent says "It is 0°C in Lahore" (it is really 35°C)
Why? ->   the get_weather tool returned 0
Why? ->   the tool code does data.get("main", {}).get("temp", 0)
Why? ->   someone added .get() to stop a KeyError: 'main'
Why? ->   the weather API replied {"cod": 401, "message": "Invalid API key"}
Why? ->   the .env file on the server has an old WEATHER_API_KEY   <- REAL CAUSE
```

Real-world examples, by situation:

A tool call failing — the symptom fix vs. the real fix:

```
# SYMPTOM FIX: the KeyError is gone, but the tool now lies with 0°C
def get_weather(city: str) -> str:
    params = {"q": city, "appid": KEY}
    data = requests.get(URL, params=params, timeout=10).json()
    return f"{data.get('main', {}).get('temp', 0)}°C"

# REAL FIX: check the response, and fail loudly with a clear message
# (Doc01 + Doc02)
def get_weather(city: str) -> str:
    params = {"q": city, "appid": KEY}
    response = requests.get(URL, params=params, timeout=10)
    if response.status_code == 401:
        raise ToolConfigError(
            "Weather API key is invalid — check WEATHER_API_KEY in .env"
        )
    response.raise_for_status()
    return f"{response.json()['main']['temp']}°C"
```

The real fix also shows the *next* person the real cause directly in the error message.

An agent that loops forever: the symptom fix is to lower the loop limit so it stops sooner. The real cause might be a tool that returns `"Error"` with no details, so the model keeps calling it again, hoping for a different result. The real fix is to return a clear error the model can act on (for example, `"City not found. Ask the user for the city name."`).

A RAG answer that is made up: the symptom fix is to add "Do not make things up" to the prompt. The real cause might be that the right chunk was never found, because the document was split in the middle of a table. The prompt change hides this; fixing the chunking solves it.

A flaky test (sometimes passes, sometimes fails): the symptom fix is to add `@pytest.mark.skip` or run it again until it passes. The real cause might be that the test depends on the order of results from a vector search, and two chunks have almost the same score.

A common mistake: stopping as soon as the error message is gone. What it causes: the bug becomes silent, and the system returns wrong data with no log line, so the next failure is much harder to trace. How to spot it: look at your fix. If it only adds `try/except`, `.get(..., default)`, `or ""`, a bigger timeout, or a prompt line like "be careful" — and you cannot say *why* the original failure happened — it is almost certainly a symptom fix.

Quick cheat sheet:

- A fix is real only when you can explain *why* the bug happened, in one sentence.
- Keep asking "why?" until you reach something in code, data, or config you can change.
- `.get(key, default)`, `except: pass`, and "be careful" in a prompt are warning signs of a symptom fix.

- In AI systems, a hidden bug usually becomes a believable wrong answer, not a crash — that is worse.
- If you must ship a quick symptom fix, log it and write down the real cause as a task.
- After the real fix, add a test that would have caught the bug (Doc13).

Reading an error trace as a map, not noise

A Python error trace (called a **traceback**) is the list of function calls that were running when the error happened, in order. The **top** is where your program started. The **bottom** is where the error actually happened. The **last line** gives the error type and message. Each block in the middle is one "frame": a file, a line number, a function name, and the line of code that was running.

Why this matters: in AI projects, tracebacks are long. They pass through your code, then through LangChain, LangGraph, `httpx`, or the OpenAI SDK — often 40 lines or more. If you scroll up and down looking for anything familiar, you waste time. If you read it in a fixed order, you usually find the right spot in under a minute: the last line first, then the last frame that is **your own code**, then upward only as far as you need to learn *why* that line got bad input.

How to read each part of a traceback:

Part of the traceback	What it tells you	What to do with it	Small example
The last line	The error type and message — *what* went wrong	Read it first, slowly, word by word	`KeyError: 'main'`
The last frame in **your** files	The line in your code that failed	Open this line first — this is usually where you start	`File "/app/tools.py", line 9, in get_weather`
Frames in `site-packages/` or `.venv/`	Library code (LangChain, httpx, openai...)	Usually skip them; the bug is rarely inside the library	`File ".../site-packages/httpx/_client.py"`
Frames above your failing line	Who called your code, and with what	Go upward only to find where the bad value came from	`result = call_tool(tool_call)` in `agent.py`
`The above exception was the direct cause of the following exception:`	Two errors are chained (someone used `raise ... from e`)	The **first** (upper) traceback is the original cause — read it	Your `ToolError` wraps the original `KeyError`

Part of the traceback	What it tells you	What to do with it	Small example
`During handling of the above exception, another exception occurred:`	A second error happened *inside* an `except` block	Fix the first error; the second one is often a bug in the error handling itself	A `logger.error(obj.name)` inside `except` when `obj` is `None`

Where you'll meet it: every time Python code crashes, from here on. You met tracebacks already in Doc02 (timeouts, `HTTPStatusError`), Doc06 (a tool crashing), and Doc09 (a node crashing inside the graph). In multi-agent systems (Doc11, Project 4), the traceback often starts deep in LangGraph's runner and only the last few frames are your agent's node — so the "find the last frame in your own files" rule is the one you use most. In production (Doc12), you read the same traceback from a log file or a tracing tool instead of your terminal, so log errors with `logger.exception(...)`, which saves the full traceback (the Doc01 logging pattern).

How it works — a real, short traceback from a tool call failing:

```
Traceback (most recent call last):
  File "/app/main.py", line 41, in <module>
    answer = run_agent("What is the weather in Lahore?")
  File "/app/agent.py", line 58, in run_agent
    result = call_tool(tool_call)
  File "/app/tools.py", line 22, in call_tool
    return TOOLS[name](**args)
  File "/app/tools.py", line 9, in get_weather
    temp = data["main"]["temp"]
KeyError: 'main'
```

Read it in this order:

1. **Last line:** `KeyError: 'main'` — a dict did not have the key `"main"`.
2. **Last frame in your code:** `tools.py`, line 9, `data["main"]["temp"]`. So `data` has no `"main"` key.
3. **Ask why `data` looks wrong:** `data` came from the weather API. Print it: `{"cod": 401, "message": "Invalid API key"}`. That is the real cause.
4. **You did not need** `main.py` or `agent.py` at all — the calls above only tell you *how* the program got there.

Real-world examples, by situation:

A LangGraph loop that never ends:

```
Traceback (most recent call last):
  File "/app/run.py", line 12, in <module>
    graph.invoke({"task": "Write a blog post"}, config={"recursion_limit": 25})
  File ".../site-packages/langgraph/pregel/main.py", line ..., in invoke
    ...
langgraph.errors.GraphRecursionError: Recursion limit of 25 reached without
hitting a stop condition.
```

There is no frame from your own node here. This tells you something important: no single line of code is broken. The *routing* is wrong — the graph kept moving between nodes and never reached `END`. Your next step is to look at the routing function (the conditional edge), not at a line number.

A chained error (you wrapped a library error with `from e`, as taught in Doc01):

```
Traceback (most recent call last):
  File "/app/tools.py", line 14, in search_docs
    results = client.search(query)
httpx.ConnectTimeout: timed out

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "/app/agent.py", line 30, in research_node
    docs = search_docs(state["question"])
  File "/app/tools.py", line 16, in search_docs
    raise SearchToolError("search service did not answer") from e
SearchToolError: search service did not answer
```

The bottom error (`SearchToolError`) is your friendly wrapper. The upper one (`httpx.ConnectTimeout`) is the real cause: the search service did not answer in time. Check the service and your timeout setting (Doc02).

A structured output that does not match the model:

```
  File "/app/triage.py", line 27, in classify_ticket
    ticket = Ticket.model_validate_json(raw)
pydantic_core._pydantic_core.ValidationError: 1 validation error for Ticket
priority
  Input should be 'low', 'medium' or 'high' [type=literal_error,
  input_value='urgent', input_type=str]
```

The last lines are very exact: the field is `priority`, the allowed values are `low/medium/high`, and the model sent `'urgent'`. The real cause is usually in the prompt or schema (the model was never told the allowed values), not in `triage.py` line 27.

A common mistake: reading only the *top* of the traceback, or copying the whole thing into a search engine. The top is where the program started (`main.py`), and it is almost never where the bug is. What it causes: you change code in the wrong file, and the bug stays. How to spot it: if you cannot say "the error type is X, and the last line of my own code that ran was Y", you have not read the traceback yet.

Quick cheat sheet:

- Read the **last line** first: error type + message.
- Then find the **last frame in your own files**, and skip `site-packages/` frames.
- Go **upward** only to learn where the bad value came from.
- With chained errors, the **first** traceback ("direct cause") is usually the real one.
- No frame from your own code (like `GraphRecursionError`) often means a routing or logic problem, not a broken line.
- In production, log errors with `logger.exception(...)` so the full traceback is saved.

Reproduce before you guess

To **reproduce** a bug means to have a reliable way to make the failure happen again, on purpose — ideally with one command. You do this *before* you form a guess about the cause. A guess tested against a bug you cannot reproduce cannot really be tested: if the error does not appear after your "fix", you cannot know whether the fix worked, or whether the bug just did not happen that time.

Why this matters so much in AI systems: normal code gives the same output for the same input. AI systems often do not. The model can pick different words each run, a vector search can return a different order when two chunks have almost the same score, an API can be slow only at busy times, and an agent can call tools in a different order. So many AI bugs "only sometimes happen". For these bugs, the first real work is not fixing anything. It is finding the *exact condition* that makes the bug happen every time: one specific input, one saved state, one tool result, or one timing.

How to make a bug reproducible, by situation:

Situation	What to do	Why	Small example
A bug that happens every time with one input	Write the smallest script or test that triggers it	One command to run = fast test of each guess	`python repro_weather.py` or `pytest tests/test_repro_42.py`

Situation	What to do	Why	Small example
The model's answer changes each run	Save the **exact** messages sent to the model, and re-send them; lower randomness (`temperature=0`)	Removes the prompt as a variable. Note: `temperature=0` makes runs more similar, but not always identical	Save `messages` to `failing_input.json` and replay it
A tool result causes the problem	Replace the real tool with a fake that returns the saved bad result	The bug happens every time, with no network and no cost	`fake_search = lambda q: SAVED_RESULT`
A graph fails only after many steps	Use the checkpointer to see the saved state at each step, and restart from the step before the failure	You do not need to run the whole graph again to reach the bug	`graph.get_state_history(config)`
It fails only in production, never locally	Compare the differences: env vars, library versions, data, load	Something is different — find it	`pip freeze` on both; log the config at startup (Doc01)
It fails "about 1 time in 10"	Run it many times in a loop and count; change one thing at a time	"Sometimes" becomes a number you can test against	Fails 9 of 100 runs before the fix, 0 of 100 after

Where you'll meet it: in every planted bug round in this document, starting with the "Real-world" rounds, which are often "only sometimes" bugs. You already have the tools: saving inputs is the run-recording idea from Doc13, fake tools are how Doc13 tests tools without the LLM, and checkpoints come from Doc09. In multi-agent systems (Doc11, Project 5) this is how you debug one agent at a time: save the shared state that was handed to the failing agent, then run *only that agent* with that state. In a real job, the same idea is how you debug a customer complaint about a support chatbot: you find the saved run for that conversation, and replay it.

How it works — turn one bad run into a script you can run again:

```
# repro_wrong_answer.py — run: python repro_wrong_answer.py
import json
from openai import OpenAI

client = OpenAI()
```

```
# The exact messages from the failing run, saved from the logs
# (Doc13 run records)
with open("failing_run_2026_09_14.json") as f:
    messages = json.load(f)["messages"]

bad_count = 0
for i in range(20):
    response = client.chat.completions.create(
        model="gpt-4o-mini", messages=messages, temperature=0
    )
    answer = response.choices[0].message.content
    if "refund" not in answer.lower():   # the thing that went wrong
        bad_count += 1

print(f"Bad answer in {bad_count} of 20 runs")
```

Now you have a number. Change one thing (the system prompt, the retrieved context, the model), run it again, and compare. A fix is proven when the number goes to 0 and stays there.

Real-world examples, by situation:

RAG returning wrong chunks — reproduce with the retriever alone:

```
# No LLM, no agent — just the search step, with the exact question
# from the bad run
question = "What is the refund period for annual plans?"
for doc in retriever.invoke(question):
    score = round(doc.metadata.get("score", 0), 3)
    print(score, doc.metadata["source"], doc.page_content[:80])
```

If the refund policy chunk is not in this list, the bug is reproduced in two seconds, and you know it is a search problem, not a model problem.

A multi-agent handoff losing state — replay only the second agent:

```
# The state the Research agent handed over, saved from the checkpointer
saved = list(graph.get_state_history(config))
state_before_writer = None
for snapshot in saved:
    if snapshot.next == ("writer",):
        state_before_writer = snapshot.values

# empty? then the bug is in the handoff
print(state_before_writer.get("research_findings"))
```

```
# run only the Writer, as many times as you want
result = writer_node(state_before_writer)
```

An agent that loops "sometimes": run the same task 20 times with `graph.stream(inputs, config, stream_mode="updates")` and print which node ran at each step. Compare a good run with a looping run. The first step where they differ (for example, the router picked `"research"` again instead of `"write"`) is the condition that makes the bug happen.

A timeout that only happens at busy times: fake it. Point the tool at a small local server that waits 30 seconds before answering, or mock the client to raise `httpx.ReadTimeout`. Now the "busy time" happens every run, and you can check your retry and timeout code from Doc02.

A common mistake: changing code *before* you can make the bug happen on purpose, then running the program once, seeing no error, and saying "fixed". What it causes: for a bug that happened 1 time in 10, one clean run means almost nothing — there was a 90% chance it would pass even with no fix. How to spot it: if you cannot answer "what exact command makes this bug happen, and how often?", you are still at the reproduce step, not the fix step.

Quick cheat sheet:

- No reliable reproduction = no real test of your fix.
- Save the exact input: messages, tool results, graph state.
- Make the repro small: one script, one test, or one node — not the whole app.
- Replace slow or random parts (APIs, search, the model) with saved or fake results.
- For "sometimes" bugs, run it many times and count; compare counts before and after the fix.
- Turn the repro into a real test when you are done (Doc13), so the bug cannot return quietly.

Debugging systems with many layers

An AI agent system is built from several **layers** stacked on top of each other: plain Python code, API calls, tools, retrieval (search), the graph (routing and state), and at the top, the model's reasoning and the final answer. When something goes wrong, every layer shows the same symptom from outside: *a wrong or missing final answer*. That answer could come from a plain Python bug, an API error that was quietly swallowed, a bad tool description that made the model choose the wrong tool, a chunking problem in search, or a state bug in the graph.

Why checking layer by layer matters more here than in normal software: the final answer is the layer *furthest* from most real causes, so it is the *least* useful place to start. People often start by changing the prompt, because the prompt is the easiest thing to change. But a better prompt cannot fix a tool that was never called, or a document that was never found. Check the lower layers first, in order. Only after you see correct inputs going into the model should you decide that the model's own reasoning is the problem.

Check the layers in this order (bottom to top):

Layer (check in this order)	Question to ask	How to check it	Typical bug here
1. Python & config	Does the code run, with the right settings?	Traceback, startup logs, print the loaded config	Wrong `.env` value, import error, `None` passed around
2. API calls	Did each outside call really succeed?	Log status codes and response bodies (Doc02)	401 bad key, 429 rate limit, a swallowed timeout
3. Tools	Was the right tool called, with the right arguments, and what did it return?	Log each tool name, arguments, and result	Wrong tool chosen; arguments in the wrong format; tool returns `"Error"`
4. Retrieval (RAG)	Were the right chunks found for this question?	Run the retriever alone with the same question	Bad chunking, wrong filter, old index, wrong embedding model
5. Graph / handoffs	Did routing go the way you expected, and was state passed correctly?	`graph.stream(..., stream_mode="updates")`, checkpoint history	Wrong edge, endless loop, a state field overwritten or never set
6. Model reasoning	Given **correct** inputs, did the model still answer badly?	Replay the exact messages alone (previous topic)	Unclear prompt, missing instruction, task too hard for the model

Where you'll meet it: this table is the checklist for the "Intermediate" and "Multi-agent" rounds in this document. You built each layer before: API calls in Doc02, tools in Doc06, retrieval in Doc08, graphs in Doc09, agents talking to each other in Doc11. The tracing from Doc13 is what makes layers 2-5 visible without adding `print()` everywhere. In real systems, this is how you debug a RAG app over company documents, a support desk with triage and reply agents, or a research pipeline like Project 4 and Project 11. The more agents you have, the more layers you have — and each agent's handoff is one more place to check.

How it works — see every layer in one run, instead of only the final answer:

```
# stream_mode="updates" gives you what each node changed, step by step
inputs = {"task": "Summarize the refund policy"}
for step in graph.stream(inputs, config, stream_mode="updates"):
    for node_name, update in step.items():
        logger.info("node=%s update_keys=%s", node_name, list(update.keys()))
        if "messages" in update:
            last = update["messages"][-1]
```

```
            tool_calls = getattr(last, "tool_calls", [])
            if tool_calls is None:
                tool_calls = []
            for call in tool_calls:
                logger.info("  tool=%s args=%s", call["name"], call["args"])
```

With this log, you can answer layers 3 and 5 right away: which nodes ran, in which order, which tools were called, and with which arguments.

Real-world examples, by situation:

RAG returning wrong chunks (the answer looks like a model problem, but it is layer 4): a user asks about the refund period and the bot says "30 days", but the policy says 14 days. Before you touch the prompt, run the retriever alone with that question. The top chunk is from `old_policy_2024.pdf`. Real cause: the old file was never removed from the index. No prompt change could fix this.

A tool call failing quietly (looks like layer 6, is really layer 2): the research agent says "I could not find any information about this company". The tool log shows `tool=web_search args={'query': 'Acme Ltd'}` and the result `[]`. Checking the API layer shows every call returned `429 Too Many Requests`, and the tool code turned that into an empty list. Real cause: rate limit, plus a tool that hides API errors. Fix: retry with backoff (Doc02) and return a clear error message to the model.

A graph looping (layer 5): the stream log shows `researcher → reviewer → researcher → reviewer ...`. The reviewer's update always has `approved: False`. The router reads `state["approved"]`, but the reviewer writes the field as `"is_approved"`. So the router never sees `True`. Real cause: a field-name mismatch between a node and the router — a graph bug, not a model bug.

A multi-agent handoff losing state (layer 5): the Writer agent keeps writing about a random topic. Checkpoint history shows `research_findings` was filled after the Research node, but it is empty before the Writer. Real cause: the Research node changed the state object in place (`state["notes"].append(...)`) and returned `{}`. In LangGraph, the dict that a node **returns** is the update that gets saved into state. Changing the input state object directly is not a supported way to update state, and those changes can be lost. A second possible cause is a list field with no reducer (a function that tells LangGraph how to combine old and new values): each update then **replaces** the old list instead of adding to it. Fix: return the update, and use a reducer like `Annotated[list, operator.add]` for fields that should grow.

```
import operator
from typing import Annotated, TypedDict

class State(TypedDict):
    task: str
    research_findings: str
```

```
    # new notes are added, not replaced
    notes: Annotated[list[str], operator.add]

def research_node(state: State) -> dict:
    findings = run_research(state["task"])
    # return the update — don't change `state` directly
    return {"research_findings": findings, "notes": ["research done"]}
```

A common mistake: starting at the top — rewriting the prompt, or switching to a "smarter" model — as soon as the final answer is wrong. What it causes: the wrong answer sometimes goes away by luck (the model guesses better), so you think it is fixed, but the real cause (a missing chunk, a failed tool, a lost state field) is still there and returns on the next question. How to spot it: if your fix is a prompt change, and you never looked at the tool calls, the retrieved chunks, or the state at each step, you skipped layers 1-5.

Quick cheat sheet:

- A wrong final answer is a symptom — it can come from any layer.
- Check from the bottom up: Python/config → API → tools → retrieval → graph/state → model.
- Log every tool name, argument, and result; stream graph updates to see routing.
- Test retrieval alone with the exact question before blaming the model.
- In LangGraph, a node must **return** its update; list fields that grow need a reducer.
- Change the prompt or the model only after you have seen correct inputs reach it.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- Python docs — pdb, the debugger — stepping through code beats guessing from `print()` statements.
- Read Anthropic — Building Effective Agents again, especially any part on failure — you now have enough built to recognize every example.

What You'll Practice

This document has no new library or tool content — it's a practiced, well-known method, applied to every kind of failure from Docs 01-13:

- **Python:** import errors, type errors, exceptions, async mistakes, package version conflicts.
- **API:** bad key, timeout, rate limit, bad request, bad JSON, login errors.

- **OpenAI:** too-long conversation, bad structured output, tool-call failure, using too many tokens, model errors.
- **LangChain:** a badly built chain, tool failure, parser failure, context problems.
- **RAG:** bad chunking, bad search results, wrong documents shown, missing context, made-up answers.
- **LangGraph:** wrong state, wrong edge, endless loop, bad routing, node failure, checkpoint problems.
- **Multi-agent:** wrong agent picked, agents disagreeing, an endless loop between agents, repeated tool calls, wrong state sharing, one agent's notes leaking into shared state, high cost, high delay.

Practice Exercises — how each round works

Where the code lives: live sessions plant bugs directly into Projects 1-4's real code, inside their own project folders (`project_1_supportdesk_chat_and_triage/` , etc.) — nothing to set up there beyond having those projects working. For the solo worked examples below, put each category's fixes and tests in `14_debugging_lab/practice/` (`mkdir -p practice`), one file per category, and run them with `pytest <file> -v` .

The full file layout, solo worked examples:

```
practice/
├── api_debugging_practice.py          API — 4 rounds
├── langchain_debugging_practice.py    LangChain — 4 rounds
├── langgraph_debugging_practice.py    LangGraph — 4 rounds
├── multi_agent_debugging_practice.py  Multi-agent — 4 rounds
├── openai_debugging_practice.py       OpenAI — 4 rounds
├── python_debugging_practice.py       Python — 4 rounds
└── rag_debugging_practice.py          RAG — 4 rounds
```

Why each script exists:

- `api_debugging_practice.py` — bugs in a copy of Doc02's retry wrapper, each tested with no network at all.
- `langchain_debugging_practice.py` — what goes *into* a prompt template, checked on the prompt alone, before any model call.
- `langgraph_debugging_practice.py` — wiring bugs (route names, state merging, loops with one exit), tested on tiny real graphs.
- `multi_agent_debugging_practice.py` — bugs *between* agents, tested at the hand-off, not inside one agent.
- `openai_debugging_practice.py` — mistakes in how the OpenAI API is used: the prompt, the schema, the size of what you send.

- `python_debugging_practice.py` — plain Python gotchas (a missing `return`, a shared default list) that look like bigger system bugs.
- `rag_debugging_practice.py` — quiet retrieval bugs (chunking, embeddings, lost sources) that look like model mistakes.

For each planted bug, you get: broken requirements, symptoms, error/log output, what should happen vs. what's actually happening. Then: **"What do you think is wrong?"** — you look into it and answer, before I tell you if you're right.

Round structure (repeated across the categories above):

1. **Basic** — a single-layer bug (like a plain Python `TypeError`), quick to find.
2. **Intermediate** — a bug that crosses two layers (like an API error looking like a parsing bug).
3. **Real-world** — a bug pulled from a realistic, production-style situation (like something that only sometimes happens).
4. **Multi-agent** — a bug that's only visible in the full Project 4 pipeline, not when testing any one agent alone.

Try one yourself first — worked examples

This document has no fixed exercise list — bugs get planted live. Before your first live session, each category below has one fully worked example, at all 4 round levels, so you can practice the Method solo first.

Jump to a category: Python · API · OpenAI · LangChain · RAG · LangGraph · Multi-agent

Python

- **Python:** See a worked example · Show me the solution

API

- **API:** See a worked example · Show me the solution

OpenAI

- **OpenAI:** See a worked example · Show me the solution

LangChain

- **LangChain:** See a worked example · Show me the solution

RAG

- **RAG:** See a worked example · Show me the solution

LangGraph

- **LangGraph:** See a worked example · Show me the solution

Multi-agent

- **Multi-agent:** See a worked example · Show me the solution

Build Task — none

This document doesn't add new code to your projects. It stress-tests everything you've already built. Bring Projects 1-4 (and the Doc12/13 API + test layer) working — bugs get planted *into* them, live, during our sessions.

Expected Behavior

- You can state a testable guess before touching any code, not just start changing things at random.
- You can tell the difference between "I fixed the symptom" and "I found the real cause" — and explain which one you did.
- After a fix, you write (or update) a test that would have caught the bug — this step isn't optional.

Test Cases

This document's "test cases" are the planted-bug rounds themselves, done live during our sessions — not something to prepare ahead of time. When you say you're ready for this document, I'll plant bugs across the categories above, one round at a time, in your actual code.

Interview Topics Preview

- Walk through a real debugging session, start to finish, out loud, as if I'm interviewing you · real cause vs. symptom fix · how you'd debug something you can't reproduce locally.

Move On When

You can work through at least 5 planted bugs across different layers (Python/API/LangGraph/multi-agent) using the method above, on your own, without me nudging you toward the answer. Full details: CURRICULUM.md §4.

This document runs live. Say **START DOCUMENT 14** when Projects 1-4 are working and you want bugs planted into them.